

Mamba

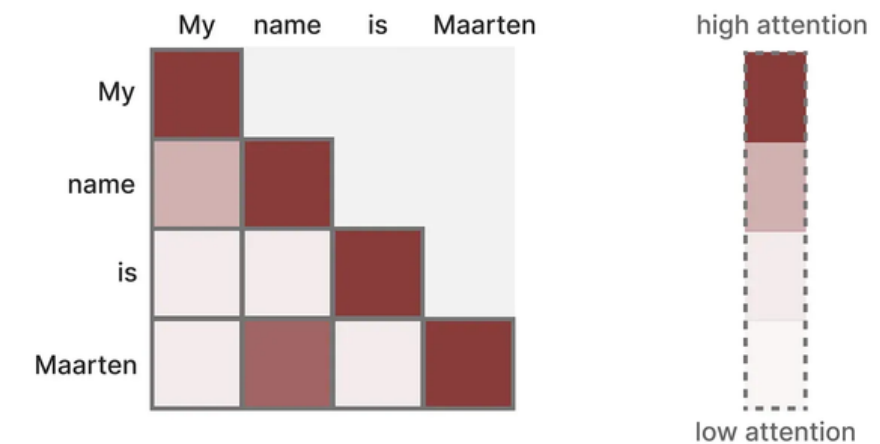
: Linear-Time Sequence Modeling with Selective State Spaces

2211606 김소영

1 Introduction

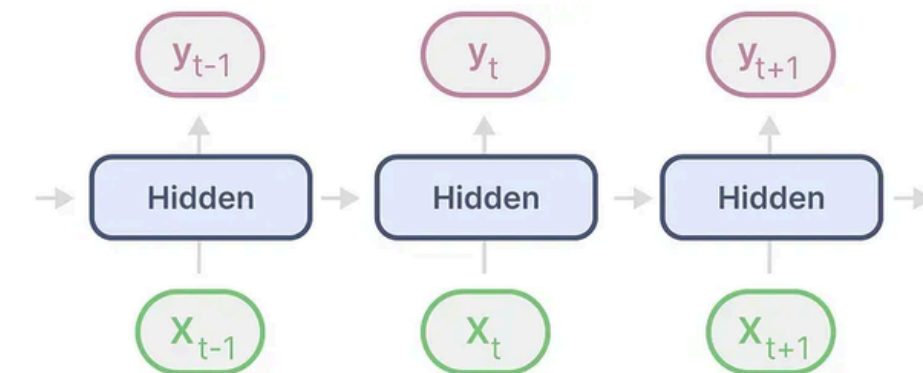
- Transformer

- 장점
 - 복잡한 문맥 모델링에 강함
 - Training Parallelism 가능
- 단점
 - Bottleneck
 - 추론 시에 다음 토큰을 생성할 때, 다시 처음부터 전체 sequence에 대한 attention 계산 필요
 - Quadratic scaling with respect to sequence length
 - 시퀀스 길이에 따라 비용 2차로 증가
- 빠른 학습, 느린 추론

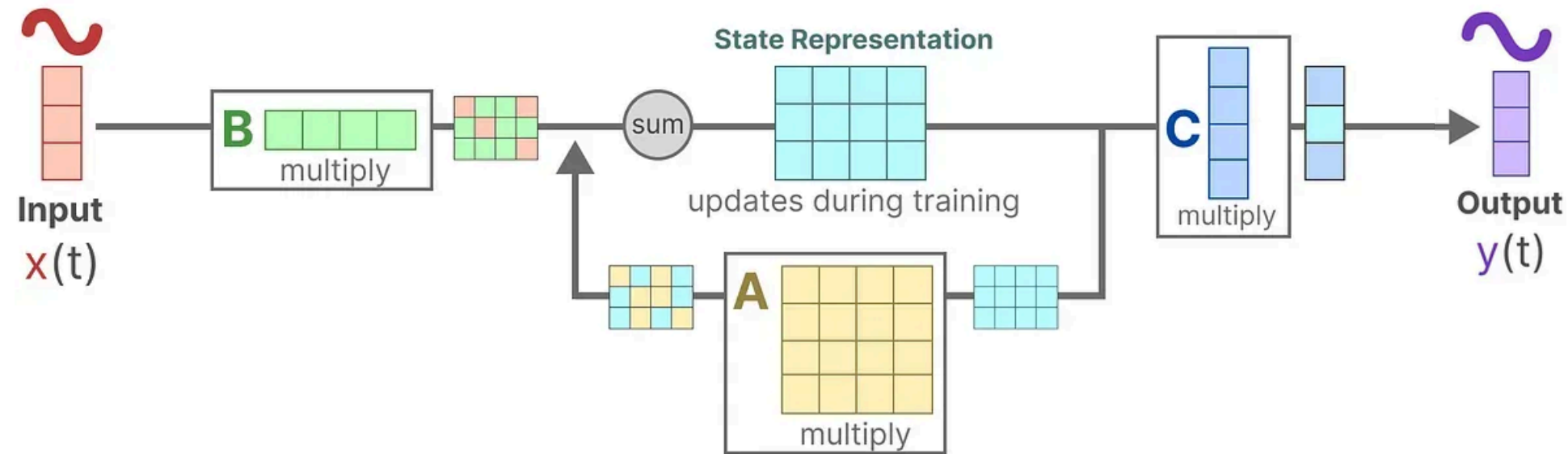


- RNN

- 장점
 - 시퀀스 길이에 따라 비용 linear하게 증가
 - 이전 state와 현재 input으로 다음 state를 생성하기 때문에, transformer와 같이 모든 이전 state들과의 연산이 필요 없음
- 단점
 - Training Parallelism 불가
 - 시간이 지남에 따라 앞의 정보를 잊어버리는 경향이 있음
- 느린 학습, 빠른 추론



2 State Space Models



- SSM
 - Mamba 모델의 뼈대를 이루고 있는 아키텍처
 - t 시점의 입력값이 들어왔을 때, hidden state를 업데이트하고 출력으로 변환하는 연속 시퀀스 모델
- **State Equation:** 상태가 어떻게 변하는지
$$h'(t) = Ah(t) + Bx(t)$$
 - A (state transition): 이전 상태가 얼마나 영향을 줄 지 = 기억력
 - B (input projection): $x(t)$ 가 얼마나 영향을 줄 지 = 입력정보 x 의 주입 강도 및 방식
- **Output Equation:** 상태가 어떻게 출력으로 변환되는지
$$y(t) = Ch(t)$$
 - C (output projection): $h(t)$ 의 어떤 부분을 출력으로 쓸 지 = 출력 생성 시 중요한 정보

2 State Space Models

Discretization

- SSM은 model based on continuous time domain, 하지만 대부분 discrete input 사용
- 흔히 다룰 수 있는 discrete한 sequence에 적용하도록 Discretization 수행

- 과정: 이산 입력 → 연속 시스템 적용 → 다시 이산 출력

1. 이산 입력을 연속 신호로 해석: **ZOH (zero-order hold)**

- 다음 이산신호를 받을 때까지 이전 이산신호 유지
- Δ : timestep, 값을 유지하는 시간 길이

2. Discretized A, B → $\bar{A}, \bar{B}$

$$\bar{A} = \exp(\Delta A), \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \Delta B$$

3. State Equation, Output Equation using $\bar{A}, \bar{B}$

- State Equation

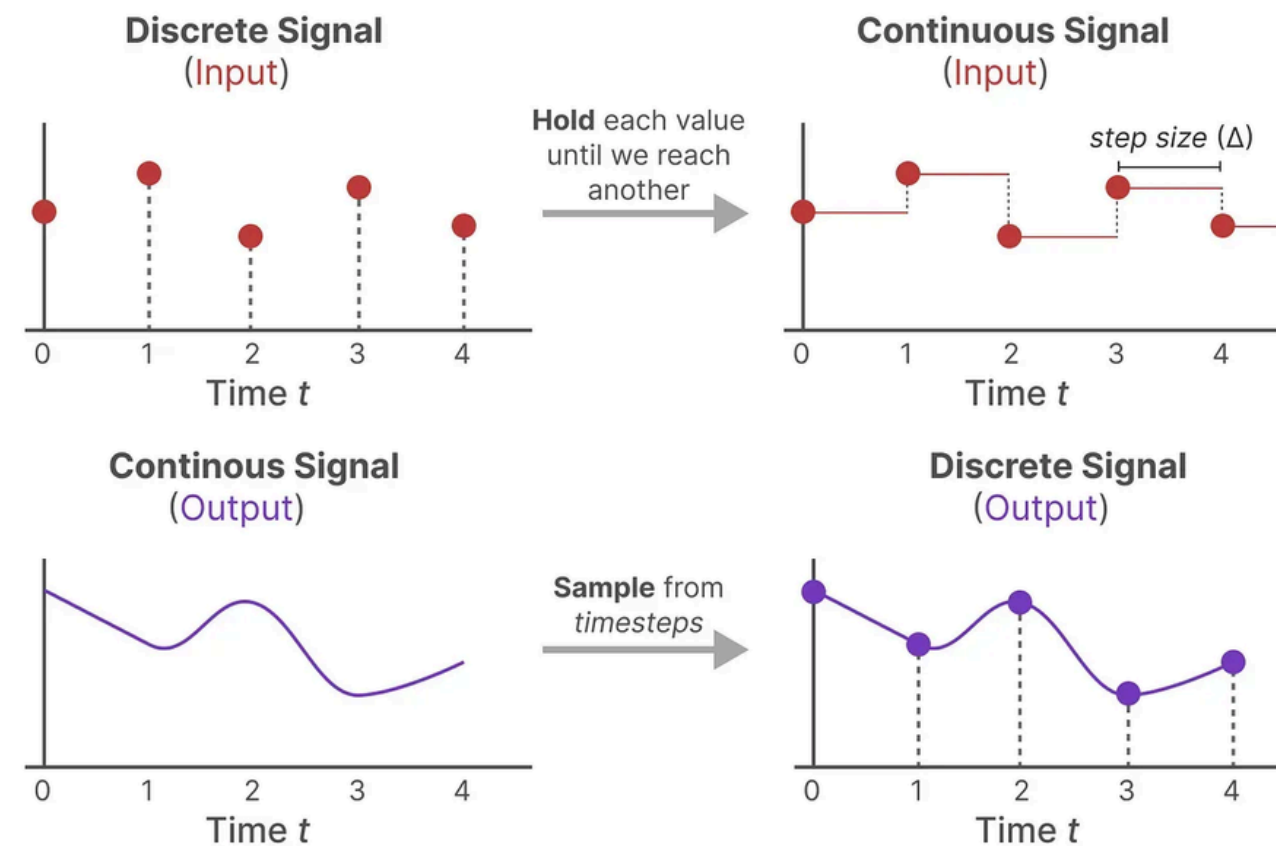
$$h_k = \bar{A}h_{k-1} + \bar{B}x_k$$

- Output Equation

$$y_k = Ch_k$$

- 다시 설명하자면,

- A: 이전 상태가 얼마나 유지/감쇠되어 전달되는 정도
- B: 현재 입력이 상태에 반영되는 정도



2 State Space Models

LTI (Linear Time Invariance)

- **Linear**
 - 상태 변화가 선형 결합으로 표현됨
 - fast training 가능
- **Time Invariance**
 - A, B, C가 timestep에 따라 변하지 않음
- Structured SSMs have all been LTI
 - 이유: convolution-based computation은 time-invariant일 때만 가능
- LTI constraint was enforced for computational efficiency
- **LTI의 한계:** time-invariant → 데이터 특성에 따라 가변적 구조를 표현하기 어려움
- Mamba
 - time-variant이면서도 computational efficiency를 유지해 bottleneck 문제를 해결해보겠다

2 State Space Models

Computation

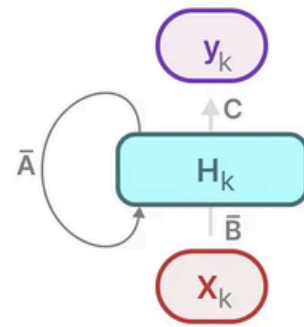
- After parameters have been transformed to 'discrete parameters', model can be computed in 2 ways

1. Linear Recurrence

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k, y_k = Ch_k$$

- 순차적으로 state update
- 병렬화 어려움

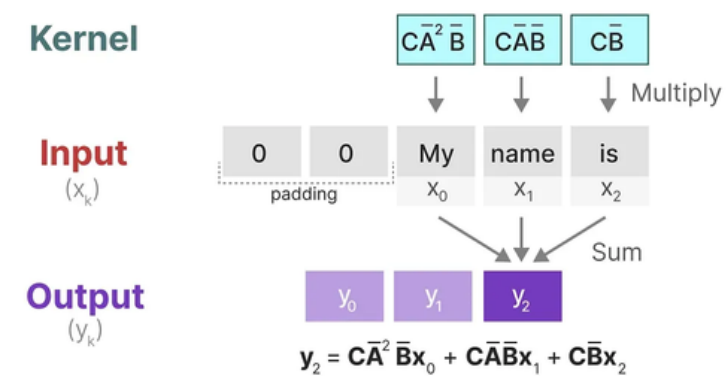
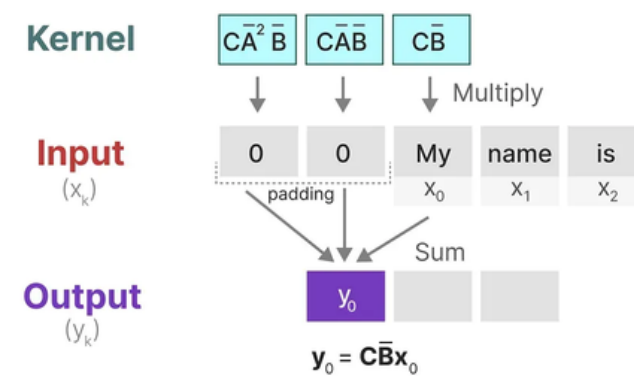
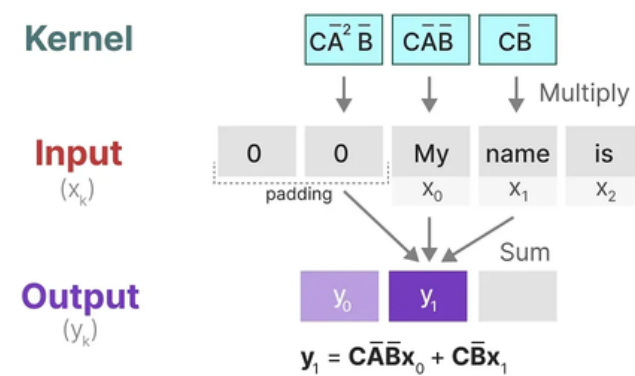
⇒ 추론 시에 사용



2. Global Convolution

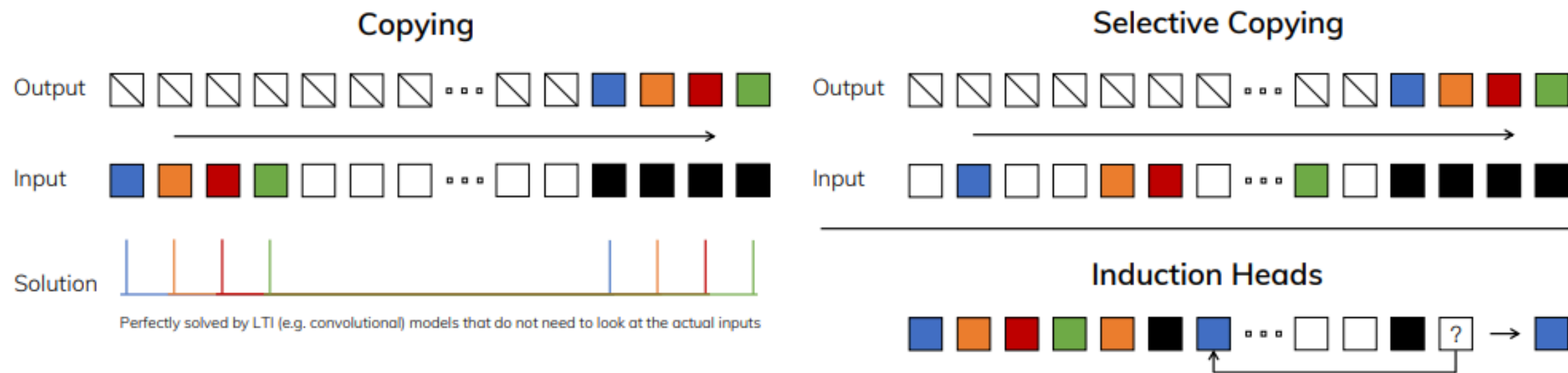
- 어떤 timestep에서 예측을 하든, 커널 내의 값은 변하지 않음
- 토큰마다 행해지는 연산이 달라지지 않기 때문에, global하게 적용될 수 있는 커널을 미리 만들어놓을 수 있음
- 전체 입력 시퀀스에 대해 한 번에 convolution 수행
- 병렬화 가능

⇒ 훈련 시에 사용



3.1 Motivation: Selection as a Means of Compression

- Sequence Modeling의 근본적인 문제: context를 작은 state로 압축하는 문제
- Efficiency vs Effectiveness Tradeoff
 - Effectiveness : small state
 - Efficiency : containing all necessary information/performance
- LTI 모델인 SSM의 문제
 - 고정된 dynamics (A,B) 사용 → 입력에 따라 달라지지 않음
 - Input-independent
 - 입력에 따라 정보 선택 불가
 - context에서 중요한 정보 선택 불가
- 제안: **Selectivity**
 - Synthetic Tasks
 1. **Selective Copying** : 입력 중 중요한 정보만 저장, 나머지는 필터링하는 작업
 2. **Induction Heads** : context 속 특정 패턴을 보고, 그 다음에 올 출력 맞추는 작업



3.2 Improving SSMs with Selection

- Selective Mechanism 구현 방법 : **“input-dependent parameter”**

- 기존 SSM

Algorithm 1 SSM (S4)

Input: $x : (B, L, D)$

Output: $y : (B, L, D)$

1: $A : (D, N) \leftarrow \text{Parameter}$

- Represents structured $N \times N$ matrix

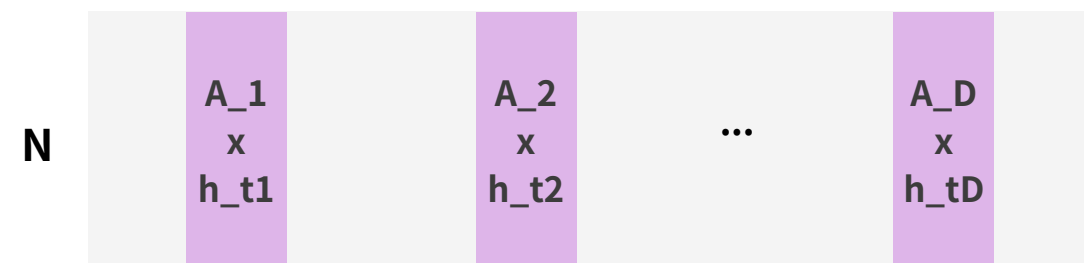
2: $B : (D, N) \leftarrow \text{Parameter}$

3: $C : (D, N) \leftarrow \text{Parameter}$ 4: $\Delta : (D) \leftarrow \tau_{\Delta}(\text{Parameter})$ 5: $\bar{A}, \bar{B} : (D, N) \leftarrow \text{discretize}(\Delta, A, B)$ 6: $y \leftarrow \text{SSM}(\overline{A}, \overline{B}, C)(x)$

- ▷ Time-invariant: recurrence or convolution

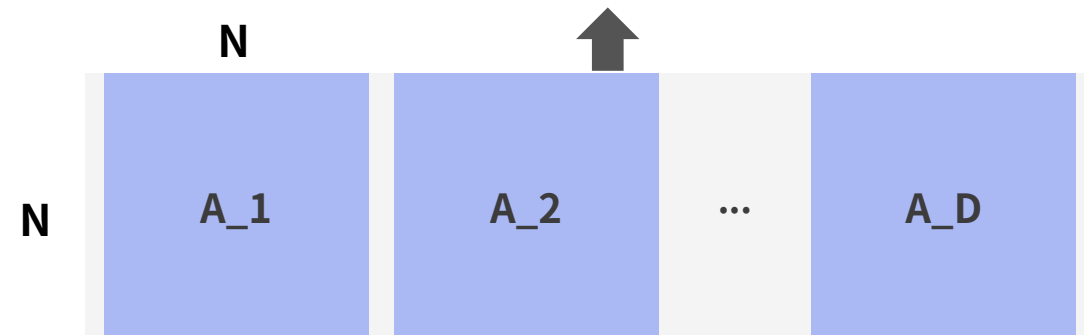
7: **return** y

< Shape of A >

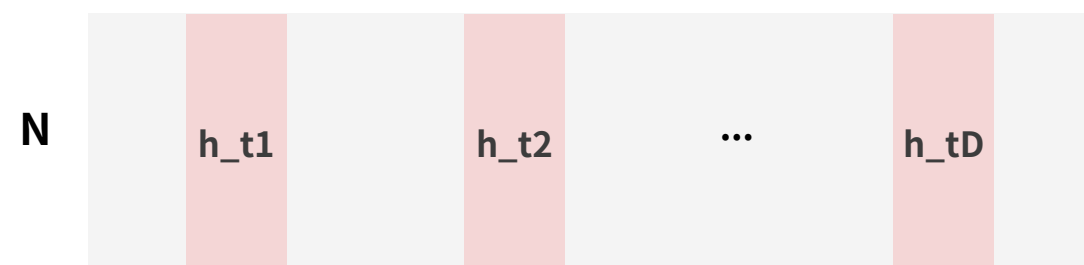


업데이트된 n차원의 hidden state를 D개를 만들어냄

A는 대각행렬이기 때문에,
A와 hidden state를 곱하는 것이 아니라
A의 대각성분과 hidden state를 곱함

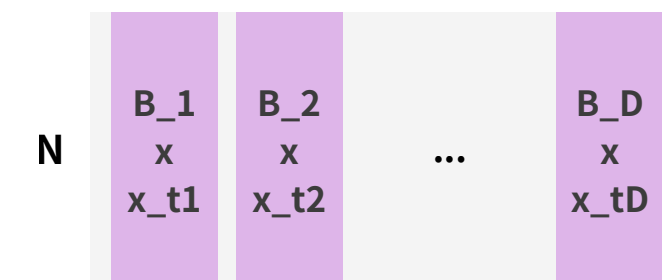


D개의 $n \times n$ matrix로
linear transform 시켜서

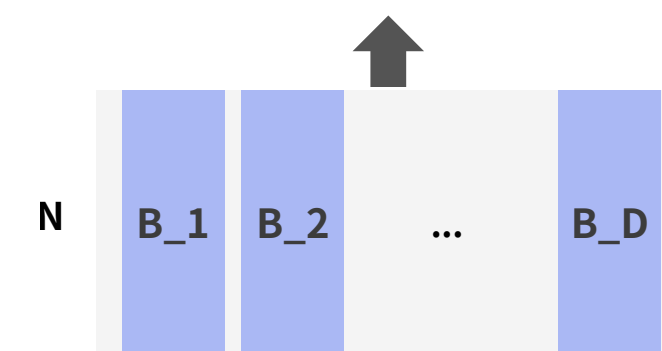


각 D개의 n차원의 hidden state를

<Shape of B>



B를 통해 각 D개의 scalar가
N차원의 Hidden space에 projection



x=D-dimesional Vector
(D개의 scalar를 가짐)



3.2 Improving SSMs with Selection

- 기존 SSM의 대안으로, input-dependent parameter 제안
- 각 batch(B)와 각 sequence(L)마다 다른 값 $\rightarrow$ shape: (B, L, ..)

- B, C, Δ 는 함수 s_B, s_C, s_Δ 연산의 결과
 - 모두 x를 input으로 하여 B, C, Δ 를 만들어내는 함수

- SSM + Selection

Algorithm 2 SSM + Selection (S6)

Input: $x : (B, L, D)$

Output: $y : (B, L, D)$

```
1:  $A : (D, N) \leftarrow \text{Parameter}$   
     $\triangleright$  Represents structured  $N \times N$  matrix  
2:  $B : (B, L, N) \leftarrow s_B(x)$   
3:  $C : (B, L, N) \leftarrow s_C(x)$   
4:  $\Delta : (B, L, D) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$   
5:  $\bar{A}, \bar{B} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, A, B)$   
6:  $y \leftarrow \text{SSM}(\bar{A}, \bar{B}, C)(x)$   
     $\triangleright$  Time-varying: recurrence (scan) only  
7: return  $y$ 
```

- s_B, s_C
 - Input: (B, L, D) 차원의 X , Output: (B, L, N) 차원
 - D차원을 N차원으로 linear mapping하는 함수
 - (B, L) 차원 추가 $\rightarrow$ 각각의 B 내에, 각각의 L 내에 적용될 수 있는 tensor가 만들어졌음
- s_Δ
 - 각 token에 대해 각각 다른 상수로 discretize
 - s_Δ 는 shape를 바꾸지 않음 (B,L,D)
 - D차원을 1차원으로 projection $\rightarrow$ 1차원을 D차원으로 broadcast $\rightarrow$ parameter 더하고, softplus
- Δ 를 통해 A와 B를 discretize
 - $\Delta: (B, L, D)$, $A: (D, N)$, $B: (B, L, N)$
 - 기존 SSM과 동일하게 ZOH로 discretize
 - 결국 discretized A, B는 data-dependent

3.3 Efficient Implementation of Selective SSMs

Motivation of Prior Models

- Efficiency vs Effectiveness Tradeoff
 - Mamba는 더이상 고정된 $\bar{A}, \bar{B}, C$ 를 사용하지 않음
 - Input-Dependent
 - 미리 global하게 적용될 수 있는 kernel 만드는 것 불가능
 - convolution을 통한 training parallelism 불가
- **Efficiency Penalty**

3.3 Efficient Implementation of Selective SSMs

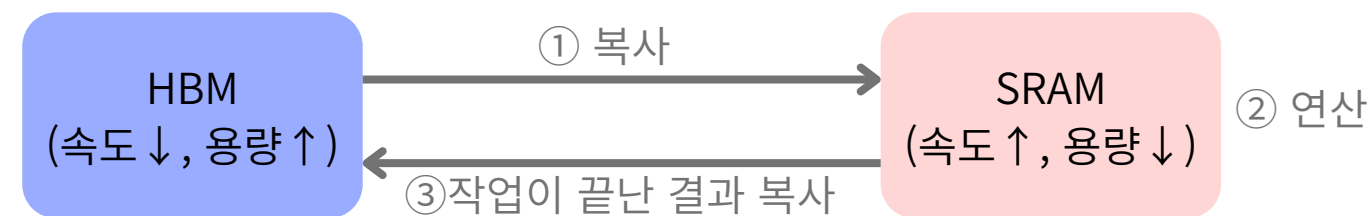
Overview of Selective Scan: Hardware-Aware State Expansion

1. Parallel Scan

- 문제: input-dependent → 미리 global kernel 생성 불가 → 병렬처리 어려움
- 조건: 연산의 associativity
 - 연산의 순서가 중요하지 않음. 연산의 순서와 무관하게 결과 동일해야 함.
- ‘먼저 계산할 수 있는 것들은 먼저 계산하자’
- 시퀀스를 부분적으로 계산하고 반복적으로 결합할 수 있음

2. Kernel Fusion

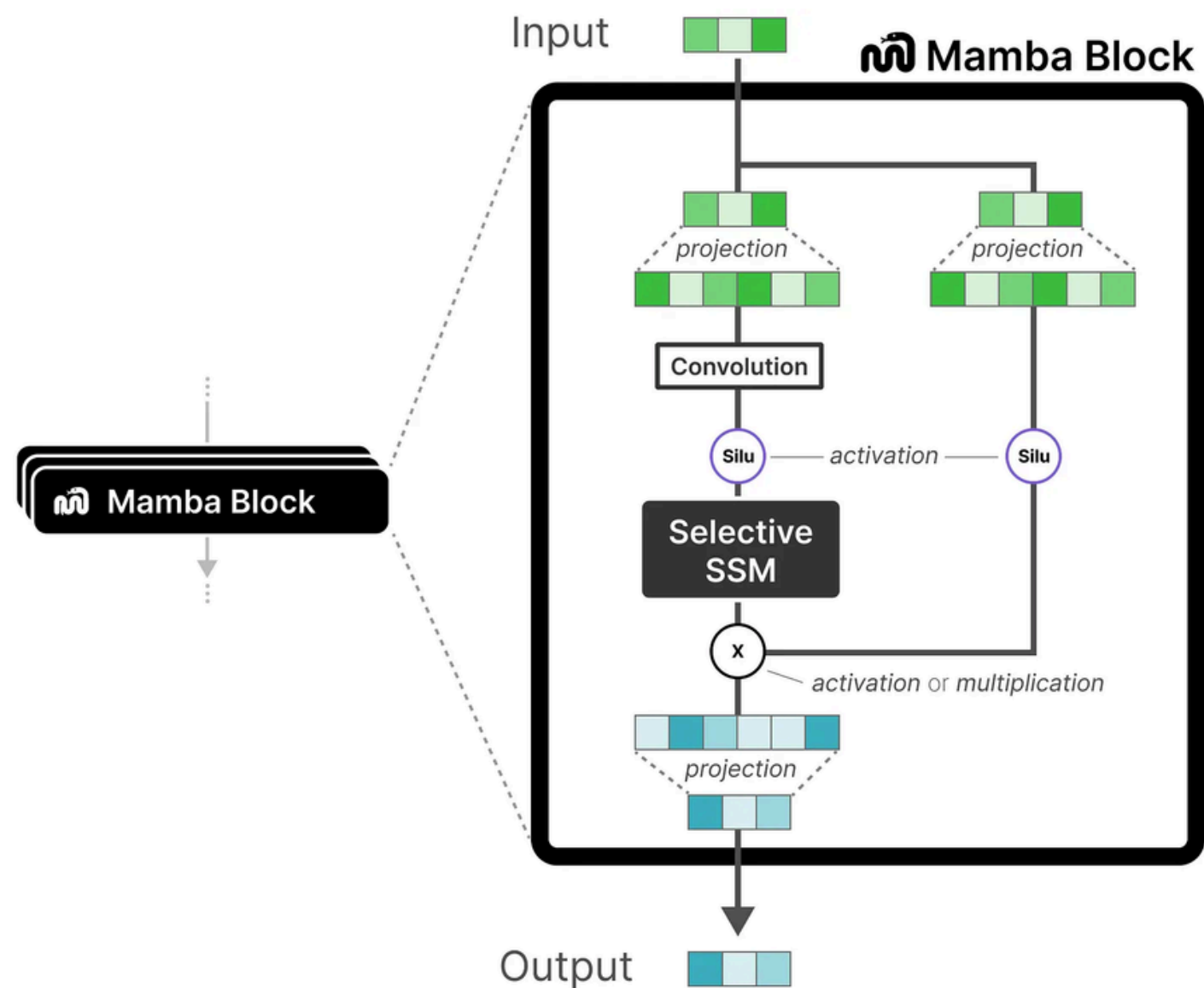
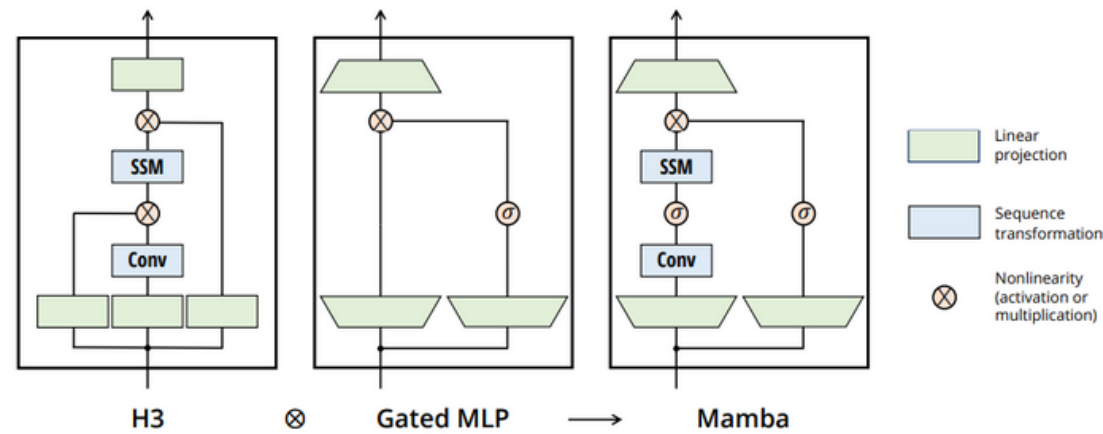
- 문제: large memory usage, memory complexity → bottleneck
 - (B,L,D)의 3차원으로 운용되는 Transformer에 비해 (B,L,D,N) 4차원의 Mamba는 GPU적 부담이 큼
 - GPU에서의 연산



- 실제 연산을 수행하는 시간보다 HBM→SRAM, SRAM→HBM으로 복사하는 시간이 더 오래 걸림
- 대안: Kernel Fusion-여러 개의 연산을 하나의 연산으로 합쳐줌
 - 문제 발생 원인: 4차원이 생기는 SSM 내부
 - 3D인 A,B,C,Δ을 HBM->SRAM으로 복사
 - SRAM 안에서 바로 discretization+recurrence 모두 수행 (4D state를 HBM에 따로 복사X)
 - SSM의 모든 연산이 끝난 이후의 결과물(3D)만 SRAM→HBM으로 복사

3.4 A Simplified SSM Architecture

Mamba Block designed by combining H3 and Gated MLP



Linear Projection

- 입력 임베딩 확장 (보통 2배)

Convolution

- 지역 정보 추출
- 같은 채널을 따라 sequence 방향으로 인접 토큰끼리 정보 교환

Selective SSM

- long-range 정보 추출
- 모든 이전 정보 기억 X. data-dependent로 선택적으로 중요한 정보만 담아서 연산.

Gating

- Conv+SSM의 결과를 gating으로 조절

Linear Projection

- 확장했던 차원을 입력 크기에 맞춰주기

3.5 Properties of Selection Mechanisms

Selective Parameter

- Δ
 - SSM 은 continuous system, Δ 의 timestep으로 이산화
 - 의미: 현재 입력에 얼마나 집중할지
 - small Δ : state 유지, 현재 입력 무시
 - large Δ : state reset, 현재 입력 강하게 반영
- A
 - 실제로 모델에 영향을 주는 건 Δ 로 이산화된 $\bar{A}$
 - Δ 만 selective으로 설정해도 충분히 성능 향상됨. A를 따로 selective하게 만들 필요 없음
- B, C
 - B: 입력 x가 hidden state에 얼마나 반영할 지를 조정
 - C: hidden state가 output에 얼마나 반영할 지를 조정

4 Empirical Evaluation

Synthetic Tasks

1. Selective Copying

MODEL	ARCH.	LAYER	ACC.
S4	No gate	S4	18.3
-	No gate	S6	97.0
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	99.7
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	99.8

Table 1: (Selective Copying.) Accuracy for combinations of architectures and inner sequence layers.

- S4: not selective / S6: selective
- model/architecture보다는 selective copying layer가 성능에 더 중요한 요인

2. Induction Heads

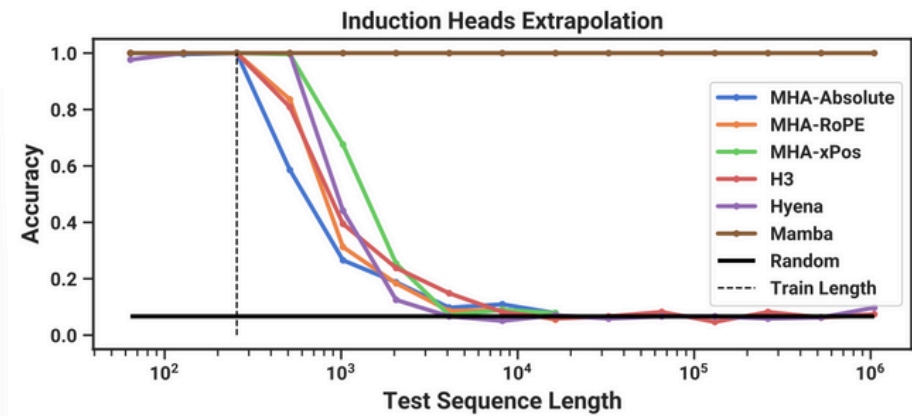


Table 2: (Induction Heads.) Models are trained on sequence length $2^8 = 256$, and tested on increasing sequence lengths of $2^6 = 64$ up to $2^{20} = 1048576$. Full numbers in Table 11.

- 추론 요구 길이를 늘려가면서 성능을 확인하는 그래프
- 학습은 고정된 길이의 시퀀스로 수행하고, 추론에서의 시퀀스 길이를 증가시킴

Transformer vs Mamba

Transformer의 문제점

- 비효율적인 긴 시퀀스 처리
- sequence의 길이가 증가하면 비용이 2차 함수 형태로 증가
- 모든 토큰을 고려하여 문맥 파악에 강하지만, long sequence를 처리할 때 bottleneck 발생 (메모리, 시간)

Mamba

- 전체 토큰이 아닌 압축된 hidden state 사용
- 기존 SSM의 문제 : time-invariant → 선택적 압축 불가
- 제안: “selectivity” ← input-dependent time-varying SSM
 - 문제: convolutionalize, 병렬 처리가 불가능 → computational efficiency ↓
 - 해결방안: Parallel Scan, Kernel Fusion

연산량(FLOPs)-성능(perplexity)

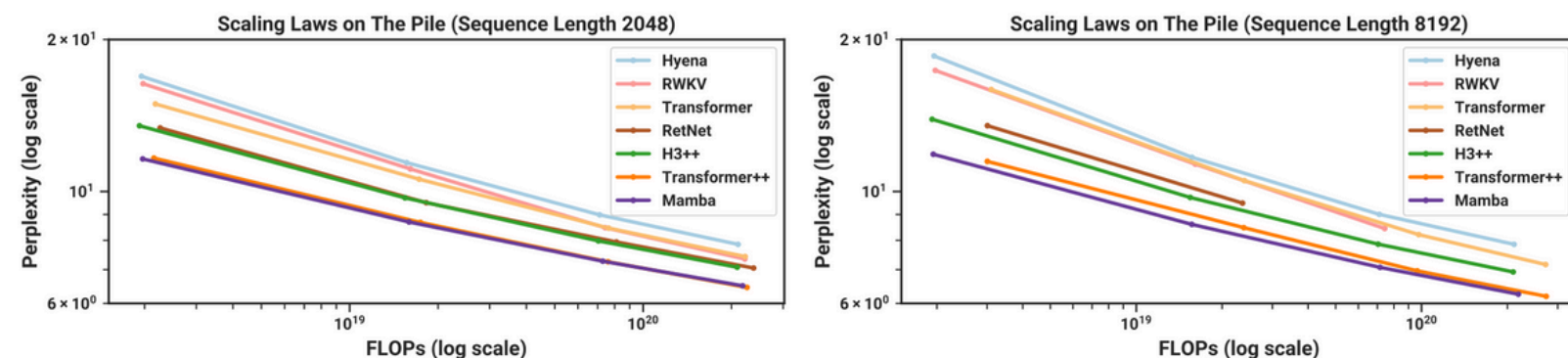


Figure 4: (**Scaling Laws.**) Models of size $\approx 125M$ to $\approx 1.3B$ parameters, trained on the Pile. Mamba scales better than all other attention-free models and is the first to match the performance of a very strong “Transformer++” recipe that has now become standard, particularly as the sequence length grows.

Efficiency Benchmarks

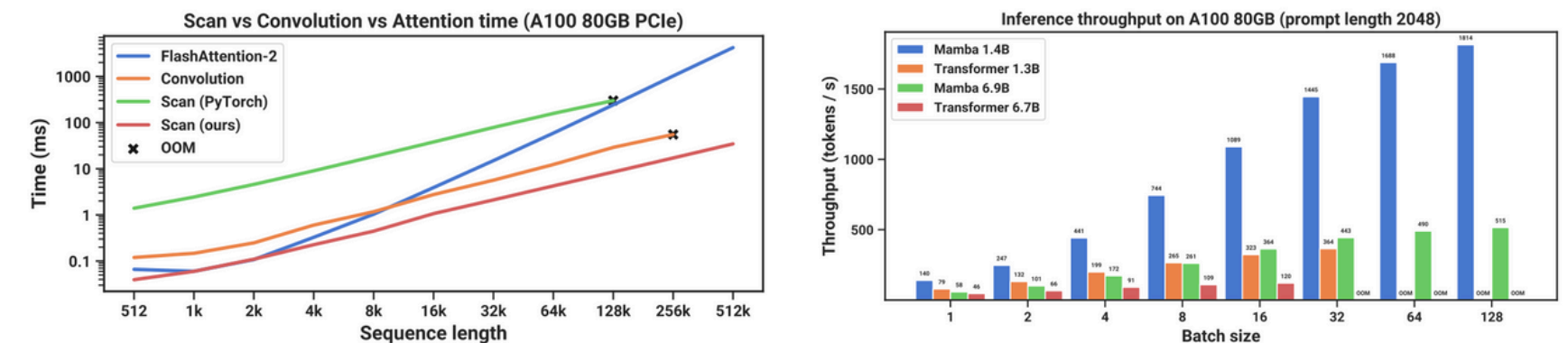


Figure 8: (**Efficiency Benchmarks.**) (Left) Training: our efficient scan is 40× faster than a standard implementation. (Right) Inference: as a recurrent model, Mamba can achieve 5× higher throughput than Transformers.

VMamba

- mamba는 기본적으로 sequential architecture
 - self-attention: 중앙 패치를 볼 때, 모든 패치를 다 봄
 - mamba: 중앙 패치를 본다고 하면, 앞에 있는 패치(파란색)는 보고, 뒤에 있는 패치(주황색)는 못 봄
 - 그 점을 보완하기 위해, 가로, 세로 방향으로 앞에서 한번 보고 뒤에서 한번 보는 식으로 총 4개의 scan 방식으로 문제점 보완

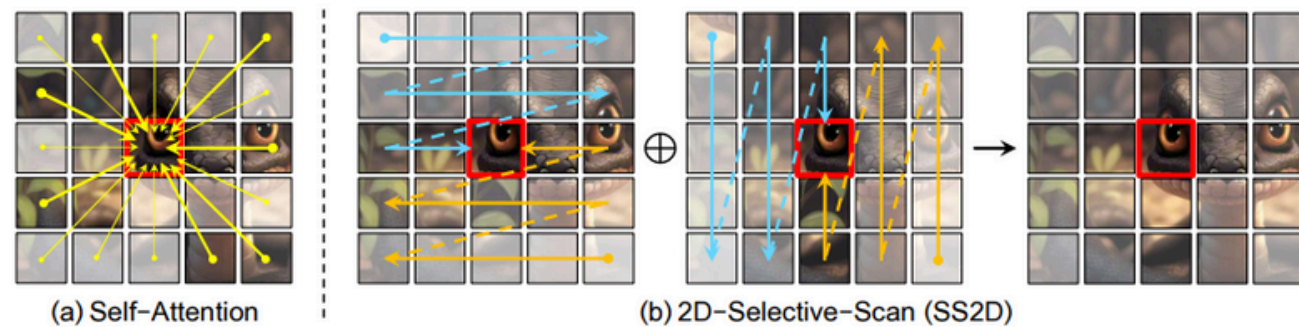


Figure 1: Comparison of the establishment of correlations between image patches through (a) self-attention and (b) the proposed 2D-Selective-Scan (SS2D). The red boxes indicate the query image patch, with its opacity representing the degree of information loss.

- SS2D block
 - 4번의 scan → S6 block (mamba) 에 넣어서 연산 → 4가지 scan merge → output

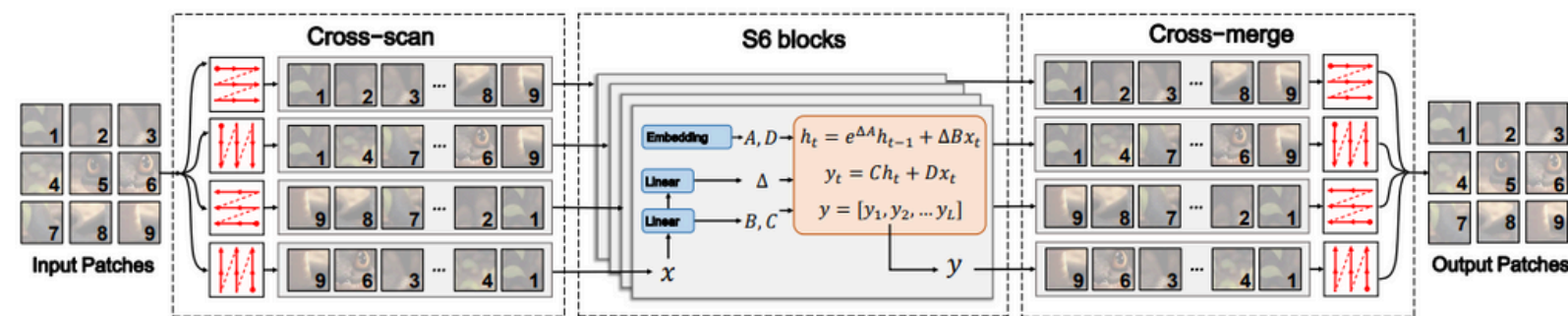


Figure 2: Illustration of 2D-Selective-Scan (SS2D). Input patches are traversed along four different scanning paths (*Cross-Scan*), with each sequence independently processed by separate S6 blocks. The results are then merged to construct a 2D feature map as the final output (*Cross-Merge*).

VMamba

- Vanilla VSS Block
 - mamba 에서 제시했던 아키텍처에서 단순히 S6 → SS2D로 바꿨는데 비효율적이었음
 - 특히 multiplicative branch가 비효율적
- VSS block
 - transformer와 같은 아키텍처로 넘어와서, 원래 attention이 있어야할 부분에 SS2D block을 사용

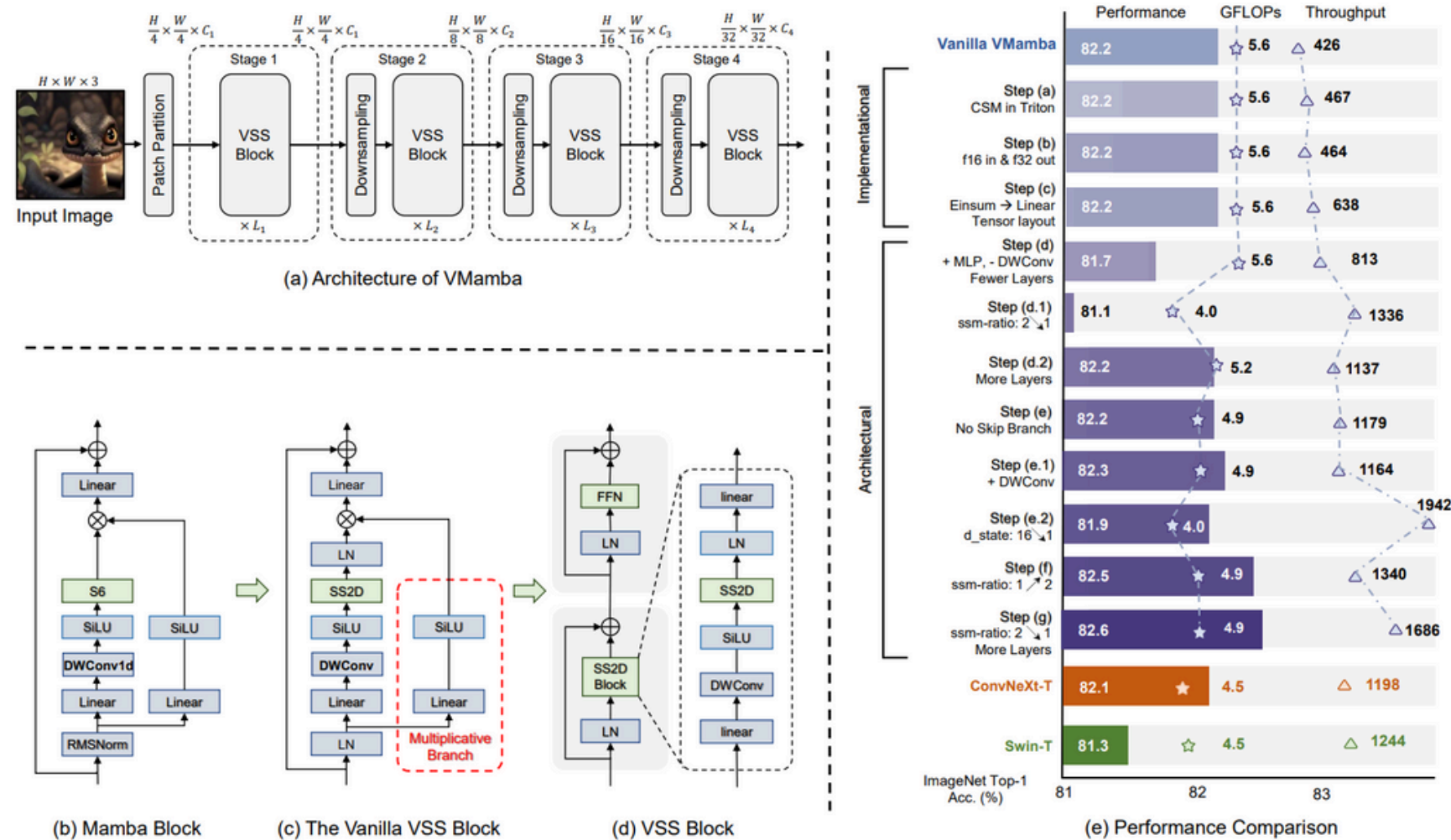


Figure 3: **Left:** Illustration of (a) the overall architecture of VMamba, and (b) - (d) the structure of Mamba and VSS blocks. **Right:** Comparison of VMamba variants and benchmark methods in terms of classification accuracy and computational efficiency.

Vision Mamba (ViM)

- 이미지 패치 변환
 - 이미지 $t(H,W,C) \rightarrow$ 패치 $x_p(J,(p^2c))$
 - (H,W) 는 이미지 크기, C 는 채널 수, p 는 이미지 패치 크기
 - 크기 D 벡터로 linear projection
 - 위치 임베딩 $E_{pos}(J+1,D)$ 추가
 - $t_{\{cls\}}$ = 클래스 토큰 사용
 - $T_{\{l-1\}}$ 이 Vim 인코더의 각 레이어에 입력 $\rightarrow T_l$ 생성
 - MLP를 거쳐 최종 prediction 생성
- **Vim Encoder**
 - 기존 mamba block에서 bidirectional하게 보도록 수정

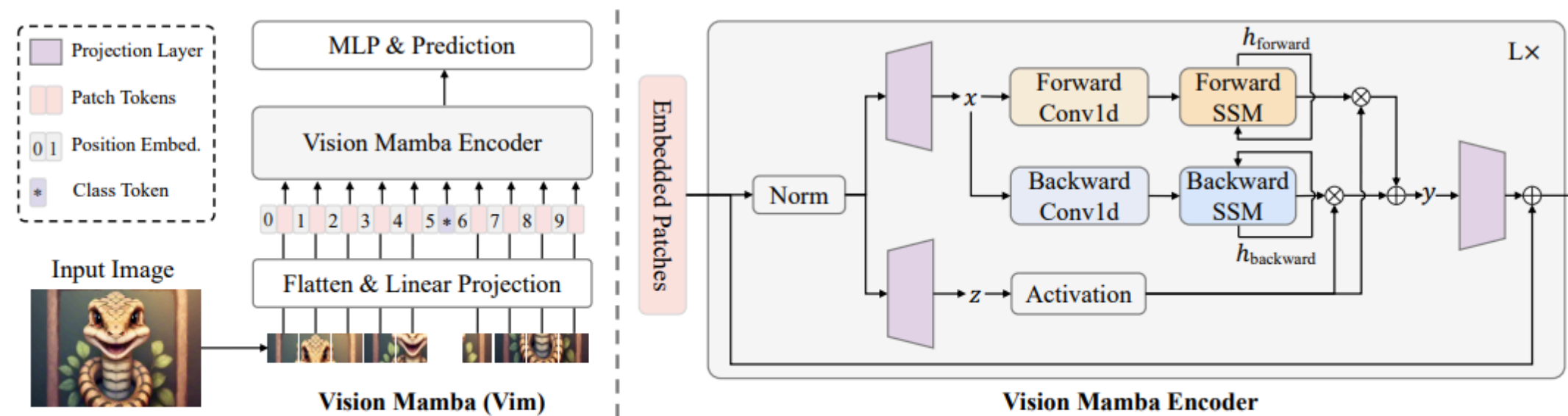


Figure 2: The overview of the proposed Vim model. We first split the input image into patches, and then project them into patch tokens. Last, we send the sequence of tokens to the proposed Vim encoder. To perform ImageNet classification, we concatenate an extra learnable classification token to the patch token sequence. Different from Mamba for text sequence